Table 3.3 Features and result helper functions of the `streamText` utility (*continued*)

Feature	Comment
Result promises	The result object also provides promises like `result.text`, `result.toolCalls`, `result.toolResults`, `result.finishReason`, and `result.usage` that resolve when all the required data is available.

While the backend response is streamed to the client, the client (or frontend, in other words) needs to be able to understand how to render the messages in chunks and without sacrificing performance with this flow of updates. This is where some additional helper functions are required in the form of React hooks. I explain them briefly next.

REACT HOOKS FOR A SEAMLESS UI

The Vercel AI SDK provides two powerful React hooks: `useChat` and `useCompletion`, which simplify the creation of conversational user interfaces and text completion capabilities in your application.

Recall that we need to update the UI to handle streaming text responses. While we could potentially modify the existing `useChatFormSubmit` hook to achieve this, the nontrivial implementation details involve managing the streaming response and keeping track of updates.

This hook should eventually handle two crucial things:

- It should handle any fragmented responses, potentially including reassembling the complete message and managing intermediate UI updates.
- Effectively displaying the streaming response requires keeping track of the current state of the message and updating the UI incrementally as new chunks arrive.

The `useChat` and `useCompletion` hooks abstract this functionality without us having to break a sweat.

Here is a quick breakdown of how those hooks work:

- `useChat`—The `useChat` hook allows you to easily create a conversational user interface for your chatbot application. "Conversation" means you can pass a list of messages as a context, and the hook will provide those as context to the AI provider. It also enables the streaming of chat messages from your AI provider, manages the state for chat input, and updates the UI automatically as new messages are received.
- `useCompletion`—The `useCompletion` hook allows you to create text completion–based capabilities for your application. This means that it will only deal with a single prompt and takes a single string as an input compared to the list of messages that `useChat` supports. You simply ask it to complete a sentence, and it will do its best to complete it.

Since we are interested in keeping a conversation with the AI model, we will be using the `useChat` hook. The basic use of this hook is explored in the following code listing.